3

演習 3　Spring Data JPA

演習の目標

■ Spring Data JPA の基本概念を理解できる

■ Controller を作成し、利用できる

■ エンティティとレポジトリーを実装できる

■ 全件検索・条件検索ができる

■ 登録・更新・削除ができる(オプション)

演習の概要

第3 章で学習したSpring Data JPA の基本文法を確認し、実際にデータベースと連動する人事管理プログラムを作成します。

■ 全社員検索機能を持つメソッドと、「全社員一覧」画面の作成■　社員番号で社員を検索する機能を持つメソッドと、画面の作成■　給与データで社員を検索する機能を持つメソッドと、画面の作成■　社員データを登録する機能を持つメソッドと、画面の作成

予想所要時間

演習3.1 10 分

演習3.2 20 分

演習3.3 15 分

演習3.4 10 分

演習3.5 25 分

■データベース情報

※データベースは、既に構築済みです。**src/main/resources/application.properties を確認してください。**

＜接続情報＞

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

＜employee テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	–	–	–
3	社員名	name	VARCHAR	–	–	–
4	給与	salary	INT	–	–	

<Employee テーブルの初期データー>

id	password	name	salary
1001	password	山田太郎	230000
1002	password	鈴木一郎	230000
1003	password	田中花子	300000
1004	password	山田次郎	400000
1005	password	高橋美智子	500000
1006	password	平恵	270000
1007	password	佐々木舞	330000
1008	password	中村翔	350000

演習3.1. Entity と Repository の定義

a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

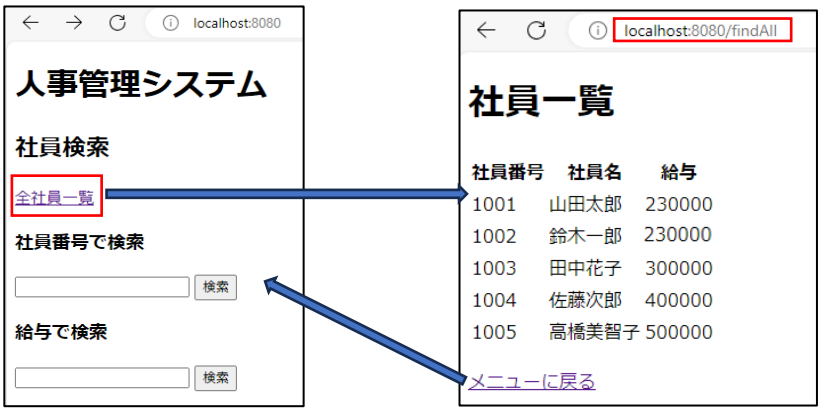
b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

演習3.2. 全社員一覧機能の実装



a. この演習では、初期画面として「社員検索」画面を利用します。この画面から、それぞれのController メソッドを呼び出します。

- src/main/resources/templates/index.html のソースコードを確認してください。

b. データベースからデータを取得したいため、Employee テーブルを操作できる EmpRepository の「依存性の注入」を行ってください。

- EmpController に戻り、EmpRepository の依存性の注入を行ってください。
- テキスト3.3.1 を参考にしてください。

c. EmpRepository を利用して、全件検索を行ってください。

- テキスト3.4 の【作業手順①】を参考にしてください。
- b.で指定したEmpRepository の変数を利用して、findAll メソッドを呼び出してください。
- 複数の行を取得するため、戻り値型はList<Employee>です。

d. 取得したリストをHTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「“empList”」にしてください。
- 遷移先は、「emp_list.html」です。

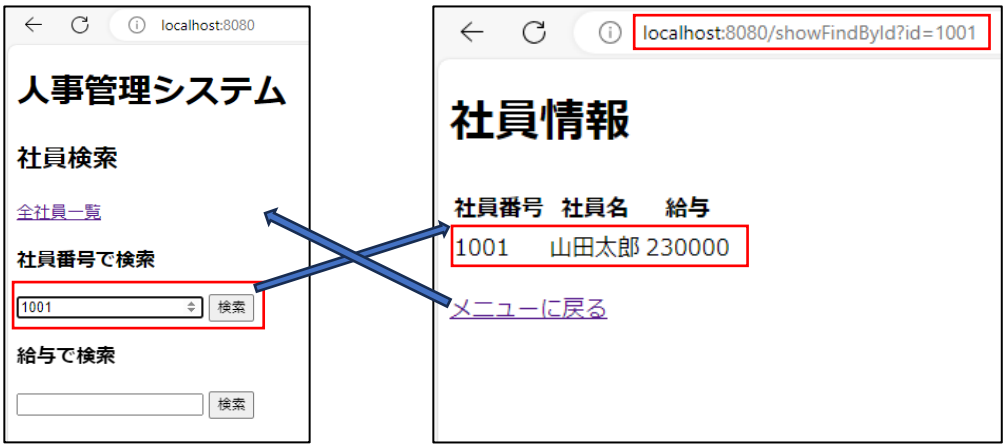
e. emp_list.html にmodel 内のデータを出力してください。

- テキスト3.4 の【作業手順②】を参考にしてください。
- パスワード以外の列を全て出力してください。
- 「:」を境に右はModel のキー名の「empList」、左はModel の中に入っているものを意味する任意の変数名にしてください。(emp など)

■ **動作確認:http://localhost:8080/**

- 「全社員一覧」リンクをクリックし、社員リストが表示されるか確認してください。

演習3.3. 社員番号検索機能の実装



a. src/main/resources/templates/index.html の「社員番号で検索」のソースコードを確認してください。

- テキスト3.5 の【作業手順①～②】が該当する内容です。
- input type = number は、整数のみ入力できるテキストフォームを生成します。
- name 属性の値を覚えてください。

b. EmpController クラスのshowFindById メソッドの引数に、a.のname 属性値を追記してください。

- テキスト3.5 の【作業手順③】を参考にしてください。

c. EmpRepository を利用して、主キー検索を行ってください。

- テキスト3.5 の【作業手順③】を参考にしてください。
- EmpRepository の変数を利用して、findById メソッドを呼び出してください。
  ※.orElse(null)は必ず付けてください。
- 1 行のデータを取得するため、戻り値型はエンティティ名のEmployee です。

d. 取得したリストをHTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「“emp”」にしてください。
- 遷移先は、「emp.html」です。

e. emp.html にmodel 内のデータを出力してください。

- パスワード以外の列を全て出力してください。
- テキスト3.5 の【作業手順④】を参考にしてください。

■ **動作確認：http://localhost:8080/**

- 「社員番号で検索」フォームに「1001」と入力し、[検索]ボタンをクリックすると、正しく社員情報ページが表示されるか確認してください。

演習3.4. 給与による社員検索機能の実装



a. src/main/resources/templates/index.html の「給与で検索」のソースコードを確認してください。

- テキスト3.5.1 の【作業手順①】の内容が該当します。
- name 属性の値を覚えてください。

b. EmpRepository にsalary 列検索用のメソッドを定義してください。

- テキスト 3.5.1 の【作業手順②】を参考にしてください。
- 複数行の検索結果を取得する可能性があるため、戻り値型を List<Employee>にします。

c. EmpController クラスのshowFindBySalary メソッドの引数に、a.のname 属性値を追記してください。

- テキスト3.5.1 の【作業手順③】を参考にしてください。

d. EmpRepository を利用して、主キーではない列(salary)検索を行ってください。

- テキスト3.5.1 の【作業手順③】を参考にしてください。
- b.で定義したメソッドを呼び出してください。
- 複数の行を取得するため、戻り値型はList<Employee>です。

e. 取得したリストをHTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「“empList”」にしてください。
- 遷移先は、「emp_list.html」です。
- emp_list.html は、演習3.2 で定義済みですので、修正事項はありません。

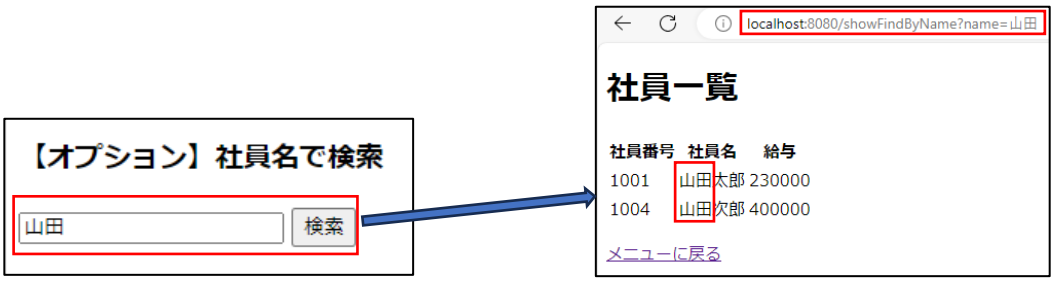
■ **動作確認：http://localhost:8080/**

- 「給与で検索」フォームに「230000」と入力し、[検索]ボタンをクリックすると、2 件の社員リストが表示されるかを確認してください。

演習3.5. 【オプション】 名前検索機能の実装

時間内に 3.4 まで完了できた方は、オプション問題に挑戦してください。

細かい修正箇所の案内はありませんので、学習した内容を基に実装してください。



a. EmpRepository に、新たにname のLike 検索を行うメソッドを定義してください。

- テキスト **3.5.3** を参考にしてください。
- 複数の行を取得するため、戻り値型はList<Employee>です。
- emp_list.html へ遷移させてください。

演習3.6. 社員データ登録機能の実装



a. EmpController のshowInputView メソッドのソースコードを確認してください。

b. src/main/resources/templates/emp_input.html のソースコードを確認してください。

- テキスト3.6 の【作業手順②】の内容が該当します。
- action 属性値が[“insertEmp”]、method 属性値が[“post”]だということを覚えてください。

- 3 つのテキストフォームがあり、それぞれname、password、salary のname 属性値が入っていることを覚えてください。

c. form.EmpForm.java を確認してください。

- テキスト3.6 の【作業手順③】の内容が該当します。

d. EmpController に戻り、メソッドの引数に、EmpForm とModel を入れてください。

- テキスト3.6 の【作業手順④】を参考にしてください。

e. Employee のエンティティをnew で生成し、Form からのデータを空っぽのEntity にコピーしてください。

- テキスト3.6 の【作業手順④】を参考にしてください。

f. EmpRepository の変数を利用して、save メソッドを呼び出してください。

- 引数には、データのコピー済みのエンティティを入れます。
- 登録処理の後は、戻り値をmodel に格納し、登録データを確認するために、result.html へ遷移させます。

■ **動作確認：http://localhost:8080/**

- [社員データ登録]リンクをクリック。
- 登録する社員データを入力し、[登録]ボタンをクリック。
- 入力したデータ通りに、新規データが登録されているかを確認。

演習3.7. 【オプション】 削除機能の実装



社員情報ページの削除ボタンをクリックすると、その社員のデータを削除する機能を実装してください。

a. emp.html の削除ボタンのコメントを外してください。

b. delete 処理をするメソッドを実装してください。

- テキスト 3.8 を参考にしてください。
- 削除処理後、全件検索の URL へリダイレクトしてください。

演習3.8. 【オプション】Service クラスの利用

テキスト 3.9 を参考にして、今回実装したメソッドのビジネスロジックを Service クラスに分離する実装に変えてください。

URL は、localhost:8080/emp2/ で実装してください。